

MagertronTM

Building an agent for Magertron

Credentials, discovery, tool calls, and delegation — for whoever writes the agent

Agent Developer Guide

DEVELOPER GUIDE · 2026-10-03 · MAGERTRONTM 4.0.x

CONFIDENTIAL — FOR EVALUATION AND OPERATOR USE

Index

Select any entry to jump straight to it. The same entries appear as bookmarks in your PDF viewer's sidebar.

What you are building	3
1 · Choose your path	3
2 · What you need from your platform team	4
3 · Five minutes: prove it works	4
4 · Calling a tool	6
Retrying a call without paying twice	6
5 · Acting for a person	7
6 · The loop	8
Cache the credential, respect the expiry	8
Fetch the catalogue as the person, not the intersection	8
Never degrade to one identity	8
Retry once on a stale 403 — and only once	9
Which refusals to retry	9
7 · A working client	10
8 · Keeping credentials and tool output safe	11
9 · When it goes wrong	12
10 · What the platform does that you do not	13

Building an agent for Magertron

This is for whoever writes the agent. It assumes your platform team has already registered your agent. §2 lists what to ask them for; everything else is yours.

Everything below has been run against a live platform. Where an error message appears, it is the exact text you will see; where part of it varies, the variable part is shown as `...`.

What you are building

Your agent calls tools through a gateway that decides, per call, whether the call may proceed. You do not implement any of that. What you implement is:

1. getting a credential — from Magertron™ itself, or from your own directory (§1),
2. discovering what you may call,
3. making calls with that credential,
4. handling the ways it can go wrong — including which refusals to retry and which never to.

The rest of this guide is those four things in order, then the safety rules (§8) and the error catalogue (§9).

1 · Choose your path

There are two ways for an agent to get its identity. Ask your platform team which one you are on; it decides §2, §3 and §5.

	PATH A Magertron™ is the issuer	PATH B Your directory is the issuer
When	No identity provider is connected to Magertron™.	Auth0, Entra ID or another OIDC directory is connected.
Your agent's identity	A service account created in the Magertron™ console. Magertron™ signs its token.	A client registered in your directory. Your directory signs its token; Magertron™ exchanges it.
What you hold	One long-lived token (1–365 days, 90 by default), issued by an admin or the developer portal and shown once.	A client ID and secret. Tokens are minted on demand and last five minutes after exchange.
Acting for a person	The token exchange (§5), with the person's token issued by an admin or the developer portal.	The token exchange (§5), with the person's token from your directory.

On Path A, a human issues every credential. Neither your agent nor the person it acts for can mint a Magertron™ token for themselves: an administrator — by hand in the console, through SCIM or Terraform — or the developer portal issues each one. That is deliberate. Every identity that can call a tool was put there by someone accountable for it.

⚠ **A Path A token is a bearer credential that works for months.** Whoever holds it *is* its principal until it expires or is revoked. §8 covers how to hold one. If your organization has a directory, Path B keeps long-lived secrets in the directory instead of in your agent.

2 · What you need from your platform team

PATH A Magertron™ as the issuer

Value	Example	What it is
Gateway URL	<code>https://mcp.acme.com</code>	where you send everything
Service-account token	<code>eyJhbGciOi...</code>	your agent's identity and its proof, in one
Person's token (<i>only if you act for someone</i>)	<code>eyJhbGciOi...</code>	one per person, issued for them by an admin or the developer portal

The token is displayed exactly once, when the service account is created. Magertron™ does not keep a copy: a lost token means a rotation, not a recovery. Ask your platform team to hand it to you through your secrets manager, not a chat message. A person's token is issued the same way, with a lifetime of 30, 60 or 90 days — and it **cannot be revoked before it expires**, so ask for the shortest lifetime that works.

PATH B Your directory as the issuer

Value	Example	What it is
Gateway URL	<code>https://mcp.acme.com</code>	where you send everything
Client ID	<code>mXaCNuzq...</code>	your agent's identity in the directory
Client secret	(64 chars)	proves it
Exchange audience	<code>https://sts.acme.com</code>	the API identifier your tokens must carry

⚠ **The audience is not a URL you call.** It is a label the directory stamps into tokens. Nothing is hosted there.

Ask them to confirm one more thing: that the API registration allows **both** client access and user-delegated access. If it allows only the first, everything in §5 will fail with a message about resource servers, and it will not be obvious why.

3 · Five minutes: prove it works

Before writing any code, make one call by hand.

PATH A Magertron™ as the issuer

```
GW=https://mcp.acme.com
SA_TOKEN=...           # yours — from your secrets manager, never pasted into history

# 1 · optional but recommended: trade the long-lived token for a five-minute one
TOKEN=$(curl -s -X POST "$GW/api/v1/token/exchange" \
  -H 'Content-Type: application/json' \
  -d '{"grant_type":"urn:ietf:params:oauth:grant-type:token-exchange",
      "subject_token":"$SA_TOKEN",
      "subject_token_type":"urn:ietf:params:oauth:token-type:jwt"}' \
  | python3 -c 'import sys,json; d=json.load(sys.stdin); print(d.get("access_token") or d)')
echo "magertron token: ${#TOKEN} chars"

# 2 · what may you call?
curl -s "$GW/api/v1/portal/catalog" -H "Authorization: Bearer $TOKEN" | head -c 400
```

You may skip step 1 and send `$SA_TOKEN` directly — it is accepted as it is. Exchanging is still the better habit: the long-lived token then travels only to the exchange endpoint, every call carries a credential that is worthless in five minutes, and a re-exchange picks up roles granted since the service account was created (§6).

PATH B Your directory as the issuer

```
GW=https://mcp.acme.com
CLIENT_ID=...          # yours
CLIENT_SECRET=...       # yours
AUDIENCE=https://sts.acme.com
IDP=https://acme.eu.auth0.com/oauth/token

# 1 · your agent's token, from your directory
AGENT=$(curl -s -X POST "$IDP" -H 'Content-Type: application/json' \
  -d '{"grant_type":"client_credentials",
      "client_id":"$CLIENT_ID",
      "client_secret":"$CLIENT_SECRET",
      "audience":"$AUDIENCE"}' \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)["access_token"])')
echo "agent token: ${#AGENT} chars"

# 2 · trade it for a Magertron credential
TOKEN=$(curl -s -X POST "$GW/api/v1/token/exchange" \
  -H 'Content-Type: application/json' \
  -d '{"grant_type":"urn:ietf:params:oauth:grant-type:token-exchange",
      "subject_token":"$AGENT",
      "subject_token_type":"urn:ietf:params:oauth:token-type:jwt"}' \
  | python3 -c 'import sys,json; d=json.load(sys.stdin); print(d.get("access_token") or d)')
echo "magertron token: ${#TOKEN} chars"

# 3 · what may you call?
curl -s "$GW/api/v1/portal/catalog" -H "Authorization: Bearer $TOKEN" | head -c 400
```

Token lengths and a catalogue mean you are connected. If an exchange printed an error object instead of a token, jump to §9 — the message names the cause.

⚠ **Check the lengths.** An empty variable produces `Authorization: Bearer` and a 401 that reads like a permissions problem. This wasted an hour of the author's day; it will waste one of yours.

4 · Calling a tool

A tool call is JSON-RPC over HTTP. Two requests: open a session, then call. This is the same on both paths.

```
NS=mcp-prod
SERVER=mags-dev-time

SID=$(curl -s -D - -o /dev/null -X POST "$GW/mcp/$NS/$SERVER/" \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json, text/event-stream' \
  -d '{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"
    "protocolVersion":"2025-03-26","capabilities":{},
    "clientInfo":{"name":"my-agent","version":"1"}}}' \
  | grep -i '^mcp-session-id:' | cut -d' ' -f2 | tr -d '\r')

curl -s -X POST "$GW/mcp/$NS/$SERVER/" \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json, text/event-stream' \
  -H "Mcp-Session-Id: $SID" \
  -H "Idempotency-Key: $(uuidgen)" \
  -d '{"jsonrpc":"2.0","id":2,"method":"tools/call","params":{"
    "name":"get_system_time","arguments":{"timezone":"UTC"}}}'
```

⚠ **Some servers are stateless** and issue no `Mcp-Session-Id`. That is not a failure. Distinguish it from a real one by the initialize response: HTTP 200 with a JSON-RPC result means stateless — carry on without the header. Anything else is a refusal, and its body says why. The reference client makes exactly this distinction:

```
if sid == "" {
    if resp.StatusCode == http.StatusOK && initHasResult(raw) {
        return statelessSession, nil // live, nothing to carry
    }
    return "", fmt.Errorf("no session id (HTTP %d): %s", resp.StatusCode, ...)
}
```

⚠ **Authenticate the handshake too.** A handshake that authenticated as one identity and a call that authenticated as two is a seam nobody can explain.

Retrying a call without paying twice

Networks fail after the vendor has done the work. If your agent retries a `tools/call` that timed out, the vendor may be paid twice and the platform will meter it twice — unless you say it is the same call.

Send an `Idempotency-Key` header on every `tools/call`: a fresh value per logical call, and the **same** value on each retry of that call. The platform meters a key once, however many times it arrives. `X-Idempotency-Key` is accepted as well; if both are present, `Idempotency-Key` wins.

⚠ **Fresh per call, stable per retry.** Reusing one key across different calls makes the platform treat them as one — under-counting, which is worse than the problem you were solving. Generate the key when you decide to make the call, before the first attempt, and keep it with the attempt.

5 · Acting for a person

Everything so far was your agent acting as itself — a complete, governed, audited call. Most agents need nothing more.

If your agent acts *for someone*, the call carries two identities and succeeds only where **both** are permitted. It can never reach further than your agent could alone; the intersection only narrows. Both paths do it the same way — the token exchange. Only the source of the two tokens differs.

The token exchange — both paths, from anywhere

Add the person's token as the subject and yours as the actor:

```
POST /api/v1/token/exchange
{
  "grant_type":      "urn:ietf:params:oauth:grant-type:token-exchange",
  "subject_token":   "<the PERSON's token>",      ← who it is FOR
  "subject_token_type": "urn:ietf:params:oauth:token-type:jwt",
  "actor_token":     "<YOUR token>",              ← who is ACTING
  "actor_token_type": "urn:ietf:params:oauth:token-type:jwt"
}
```

⚠ **The RFC's names read backwards.** The *subject* is the human; the *actor* is the agent. Reversing them produces a call that looks delegated and is not.

	PATH A subject / actor	PATH B subject / actor
subject_token	the person's Magertron™ token (§2)	the person's access token from your directory
actor_token	your service-account token	your agent's client-credentials token

⚠ **PATH B** The person's token must also carry your exchange audience. If you get it from your directory, ask for a token *for the API* — not an `id_token`. An `id_token`'s audience is the client application and the exchange refuses it:

```
token verification failed: token doesn't contain the required audience
```

The result is an ordinary bearer token, valid **five minutes**, carrying both identities signed into it. Use it exactly as in §4. You cannot change who you act for without exchanging again — which is the point.

The header form — in-cluster daemons only

There is a second form, for one case: a daemon running inside the cluster that signs people in itself, such as the reference chat daemon. It sends both credentials on every request instead of exchanging:

```
Authorization: Bearer <YOUR token>
X-Magertron-On-Behalf-Of: Bearer <the PERSON's token>
```

⚠ **Prefer the exchange.** With the header form the binding between the two identities is asserted by your agent on each call rather than signed by the platform once, and your agent holds the person's session. If you are not writing an in-cluster daemon, you do not need it.

On both paths, an agent may not act for an administrator. That delegation is refused outright rather than reduced:

```
403 on-behalf-of principal 'ops-admin' holds a non-delegable role (system:platform-admin): an agent
may not act for an administrator
```

6 • The loop

Five rules. The first three are easy; the fourth is the one that bites; the fifth is the one that costs money.

Cache the credential, respect the expiry

An exchanged token lasts five minutes, and re-exchanging per call is wasteful. Refresh on a margin:

```
// A 30-second margin: enough that a token cannot expire in flight between
// this check and the upstream reading it.
if t.cached != "" && time.Now().Add(30*time.Second).Before(t.expires) {
    return t.cached, nil
}
```

On Path A, also watch the long-lived token's own expiry. It does not renew itself; when it lapses, every exchange fails until your platform team rotates it. Alert well before the date — the console records an owner for expiry reminders, but your agent should not depend on someone reading email.

Fetch the catalogue as the person, not the intersection

If you are delegating, ask "what can this person do?" with their credential alone. Answering with the intersection hides tools your agent happens to lack, which is a confusing answer to "what can I do here" — and every call is gated anyway, so nothing becomes reachable by being listed.

Never degrade to one identity

If the delegation cannot be established, fail the call. Falling back to "just the agent" is a privilege change in the wrong direction and looks identical to the feature working.

Retry once on a stale 403 — and only once

⚠ **This is the least obvious property of the system.** A token carries the roles its principal held when it was minted. A permission granted an hour ago does not reach a token issued two hours ago, and there is no push notification — the gateway validates a signature rather than consulting a session table.

So an agent running for hours will keep being refused for a permission it now has, and the refusal looks like a platform fault. The refusal that can be stale is the one that names roles:

```
No permission for ... Principal '...' has roles [...]
```

On that message, and only that one, re-exchange once and retry:

```
if resp.StatusCode == 403 && strings.Contains(body, "No permission for") && !alreadyRetried {
    tok, _ = provider.Refresh(ctx) // re-exchange: roles are read fresh
    alreadyRetried = true
    continue
}
```

Once. An agent that retries a genuine denial indefinitely turns one refusal into a loop against the gateway. And on Path A, this only works if you exchange (§3): a raw service-account token keeps the roles it was created with.

⚠ It is easy to get this wrong while appearing correct. The reference daemon re-fetches its tool catalogue every turn, deliberately, with a comment saying entitlements can change. It still went stale — the *fetch* was fresh and the *identity* was not.

Which refusals to retry

Most refusals are decisions. Retrying a decision does not change it; it only adds load and noise to the audit trail.

Refusal	Retry?	Why
403 No permission for ...	Once, after re-exchanging	Your roles may have changed since the token was minted.
429 PDP at capacity ...	Yes, with backoff	Load shedding — no decision was made. The message says so.
429 Spend limit reached ... / Budget threshold reached ...	No	A human must lift the limit. Retrying cannot succeed and should page someone instead.
403 tool-approval: ...	No	The tool's definition is waiting for review. Calls resume when a human approves it.
403 ... holds a non-delegable role ...	No	Permanent by design.
Network error or timeout on tools/call	Yes, same Idempotency-Key	\$4: the platform meters the key once.

7 · A working client

The reference implementation is `services/mcp-chat-daemon/magertron.go`. Its shape, if you are writing your own:

Method	Does
<code>Login(user, pass)</code>	a person's session, when an in-cluster daemon signs someone in (header form)
<code>Catalog(userJWT)</code>	what may be called — the person's token alone
<code>OpenSession(..., ns, server)</code>	<code>initialize</code> ; returns a session id or the stateless sentinel
<code>CallTool(..., sessionID, tool, args)</code>	<code>tools/call</code> , with the session id when there is one and an idempotency key always
<code>SAToken(ctx)</code>	the credential: cached, refreshed, exchanged (Path B, or Path A exchanging) or static (Path A raw)

One detail worth copying: the principal headers are set by a single helper, never inline per call site, because the identities must agree on every request.

```
func setPrincipalHeaders(req *http.Request, saJWT, userJWT, turnID string) {
    if turnID != "" && sendTurnID {
        req.Header.Set("X-Magertron-Turn-Id", turnID)
    }
    if saJWT != "" {
        req.Header.Set("Authorization", "Bearer "+saJWT)
        req.Header.Set("X-Magertron-On-Behalf-Of", "Bearer "+userJWT)
        return
    }
    req.Header.Set("Authorization", "Bearer "+userJWT)
}
```

⚠ That header form is the in-cluster shape (§5), for a daemon that signed the person in itself and holds both tokens. If your agent runs anywhere else, use the exchange (§5): the binding is then established by the platform and signed, rather than asserted by you on each call, and you never hold anyone's session.

`X-Magertron-Turn-Id` is optional. Send a stable id per conversational turn and the platform groups a model call and the tool calls it produced into one unit — which is what makes a chargeback line read as an answer rather than as five unrelated calls.

8 · Keeping credentials and tool output safe

The gateway decides whether each call may proceed. It cannot stop your agent from leaking the credential that makes the call, or from being talked into making it. Those are yours.

Never let a token reach a log, a prompt or a tool

- Do not log request headers. Log the token's length or its `jti`, never the token.
- Never put a token in a model prompt, a tool argument, or anything a tool returns to the model. A model that has seen a token can repeat it.
- Never forward the exchanged token, or the person's token, to an upstream service yourself. The platform attaches vendor credentials upstream (§10); there is nothing for you to pass on.

Hold the long-lived credential like a password

- **Path A:** the service-account token lives in a secrets manager and is loaded at start-up — never in source, an image, or an environment file committed to git. If it leaks, ask your platform team to revoke it and rotate: the old token stops working and you receive a new one, once.
- **Path B:** the client secret gets the same treatment. Rotating it in your directory does not touch Magertron™.
- **A person's token is never your agent's identity.** Use it only as the `subject_token` of an exchange. Sent on its own, it makes your agent indistinguishable from the person: the audit loses the agent and the call is no longer the intersection of the two. On Path A it also cannot be revoked before it expires — hold it like the service-account token, and drop it when the work for that person is done.

Treat tool results as untrusted input

Whatever a tool returns — a web page, a document, a ticket — goes back through your model. If it contains instructions, your model may follow them, and the call it then makes will be *permitted*: the gateway checks who is calling and what they may do, not whether the request came from the user or from text inside a search result. Keep tool output clearly separated from instructions in your prompts, confirm with the person before calls that change or spend something, and do not give an agent roles it does not need — the intersection in §5 only narrows what you start with.

9 · When it goes wrong

Each of these points somewhere other than its cause. That is why they are here.

```
token doesn't contain the required audience
```

PATH B You sent an `id_token`. Ask your directory for an access token for the exchange audience. §5.

```
Client "..." is not authorized to access resource server "..."
```

PATH B The API registration allows one access type and you need both. Your platform team's fix, not yours — send them §2.

```
No service account is provisioned for subject '...@clients'
```

PATH B Your token verified; nobody recognizes the identifier. Your `externalId` in Magertron™ must equal the `sub` your directory issues — Auth0 uses `<client_id>@clients`, Entra the service principal's object ID.

```
No user is provisioned for subject 'auth0|...'
```

Same, for the person you are acting for. They must have signed in through the directory at least once, or been provisioned by SCIM.

```
The service account for '...' has been revoked.
```

Your credential was rotated out. On Path B, re-exchange; if it persists, your identity was deprovisioned. On Path A, ask for the new token — the old one will never work again.

```
No permission for MCPServer/ViewLogs in 'mcp-prod'. Principal '...' has roles [...]
```

Identity is fine, entitlements are not. The message names what was required and what you hold. A freshly provisioned identity arrives with no roles at all — that is expected, and someone grants them. If they were granted recently, §6: re-exchange once.

```
tool-approval: ...
```

HTTP 403. The tool's definition has changed since it was approved, or has not been approved yet, and a human must review it. Your permissions are not the problem and retrying will not help. The text after the colon names the tool's state.

```
Spend limit reached for <ns>/<server>: usage $... has reached the $... limit; a platform-admin override is required to resume traffic
```

HTTP 429. The server's spend limit is exhausted. A contract with a commitment reports it as `Budget threshold reached for ...` instead. Do not retry; surface it to a person.

PDP at capacity – request shed before evaluation. This is load shedding, not a policy decision; retry shortly.

HTTP 429. The only 429 you should retry — with backoff and jitter, not a tight loop.

A 401 with an empty-looking request

Check your token variable is non-empty before blaming the platform.

Everything works, but the audit shows one identity

The delegation was not carried. Either you used a plain agent token where an exchanged one was intended, or a header was empty. Nothing degrades at the point of the check — but an agent that forgets to delegate makes a perfectly valid direct call instead, and no error says so.

A call you retried appears twice in the chargeback

The retries carried different idempotency keys, or none. §4.

10 · What the platform does that you do not

Worth knowing, so you do not build it yourself:

- **Credential injection.** You never see the vendor's API key. Your agent presents its own identity; the gateway attaches the credential upstream.
- **Tool approval.** If a vendor changes a tool's schema, calls stop until a human reviews the difference — regardless of your permissions.
- **Metering and chargeback.** Every call is attributed to the identities on it. Delegated calls bill the person, with your agent shown alongside ("*person* · via *agent*"), not the agent alone.
- **Exactly-once metering.** Retries that carry the same idempotency key are metered once.
- **Budget.** A call that would exceed the contract's ceiling is refused with a 429, not billed and apologised for later.

None of this requires anything from your code beyond presenting a valid credential, sending an idempotency key, and handling a refusal gracefully.